

인공지능과

데이터 분석과 시각화

2일차



1. OpenAPI 실시간 호출 및 데이터 수집

예제 1: requests 라이브러리를 활용한 기본 웹 서버 호출

개념: 파이썬의 표준 웹 통신 패키지인 requests를 사용해 특정 웹 페이지의 주소(URL)로 데이터 전송 요청(GET)을 보냅니다.

프롬프트 입력해보기:

"파이썬 requests 라이브러리를 사용해서 'https://jsonplaceholder.typicode.com/posts/1' 웹 서버 주소에 데이터를 요청하고, 그 결과로 돌려받은 문자열(response.text)을 화면에 출력하는 간단한 코드를 작성해줘."

AI가 짜준 코드 실행해보기:

```
import requests # 인터넷 연결을 확인하기 위한 테스트용 웹 주소 호출
url = "https://jsonplaceholder.typicode.com/posts/1"
response = requests.get(url) # 서버가 응답한 원본 텍스트 출력
print("--- 웹 서버 응답 원본 ---")
print(response.text)
```

예제 2: HTTP 응답 상태 코드(status_code) 확인 및 예외 처리

개념: 서버가 요청을 성공적으로 처리했는지(200), 주소가 없는지(404), 서버 장애가 났는지(500) 등의 상태 코드 값을 확인해 안전하게 프로그램을 방어합니다.

프롬프트 입력해보기:

"웹 API를 호출한 후 응답 상태 코드가 200이면 '성공', 404이면 '유효하지 않은 주소', 그 외에는 '서버 오류'라고 출력하도록 응답 상태 코드(status_code)를 판단하는 조건문 코드를 작성해줘. 테스트 주소는 존재하지 않는 가상의 주소로 설정해줘."

AI가 짜준 코드 실행해보기:

```
import requests
url = "https://jsonplaceholder.typicode.com/posts/invalid\_url"
response = requests.get(url)
print(f"응답 상태 코드: {response.status_code}")
if response.status_code == 200:
    print("성공적으로 호출되었습니다.")
elif response.status_code == 404:
    print("경고: 호출 주소가 유효하지 않습니다.")
else:
    print("서버 오류가 발생했습니다.")
```



예제 3: 웹 데이터에서 JSON 포맷 파싱 및 딕셔너리 변환 (response.json())

개념: 현대 웹 API의 표준 데이터 형식인 JSON(JavaScript Object Notation) 텍스트를 파이썬의 딕셔너리(dict) 타입으로 자동 파싱합니다.

프롬프트 입력해보기:

"파이썬 requests 라이브러리로 'https://jsonplaceholder.typicode.com/posts/1'에서 데이터를 가져온 다음, 그 결과 텍스트를 JSON 형태의 딕셔너리로 변환(response.json())해줘. 그리고 딕셔너리에서 'title'과 'userId' 키값을 추출해 화면에 출력해줘."

AI가 짜준 코드 실행해보기:

```
import requests
url = "https://jsonplaceholder.typicode.com/posts/1"
response = requests.get(url)
# json() 메소드를 사용해 딕셔너리로 변환
data = response.json()
print(f"데이터 타입: {type(data)}")
print(f"글 제목: {data['title']}")
print(f"작성자 ID: {data['userId']}")
```

예제 4: API Query Parameter(파라미터)를 조립한 주소 호출

개념: 기상 상태나 날짜 등 특정 조건에 맞는 데이터만 필터링해서 받기 위해 주소 뒤에 파라미터(?key=value)를 바인딩하여 요청합니다.

프롬프트 입력해보기:

"위도 37.5665, 경도 126.9780인 서울 지역의 기상 예측 무료 OpenAPI('https://api.open-meteo.com/v1/forecast')를 호출하고 싶어. requests.get을 쓸 때 파라미터(params)로 위도(latitude), 경도(longitude), 그리고 시간별 온도(hourly='temperature_2m')를 지정해 호출 주소를 조립하는 코드를 작성해줘."

AI가 짜준 코드 실행해보기:

```
import requests
# 서울(위도 37.5665, 경도 126.9780) 기상청 예측용 무료 OpenAPI 주소
url = "https://api.open-meteo.com/v1/forecast"
# 쿼리 파라미터를 딕셔너리로 정의
params = {
    'latitude': 37.5665,
    'longitude': 126.9780,
    'hourly': 'temperature_2m' # 시간대별 2m 높이 온도 요청
}
response = requests.get(url, params=params)
print("호출된 전체 주소:", response.url)
```

예제 5: 중첩된(Nested) JSON 데이터 구조에서 원하는 데이터만 파싱하기

개념: 딕셔너리 내부의 리스트, 혹은 리스트 내부의 딕셔너리 구조로 꼬여 있는 깊은 API 응답 구조에서 실제 원하는 컬럼 정보만 발췌해 냅니다.

프롬프트 입력해보기:

"앞서 기상 예측 API에서 받아온 JSON 딕셔너리 데이터에서, 'hourly' 내부에 들어있는 'time' 리스트와 'temperature_2m' 리스트를 찾아 파싱하고 상위 5개씩만 추출해 화면에 출력하는 코드를 만들어줘."

AI가 짜준 코드 실행해보기:

```
# 기상청 날씨 JSON 응답 파싱
weather_data = response.json()
# hourly -> time, temperature_2m 추출
hourly_time = weather_data['hourly']['time']
hourly_temp = weather_data['hourly']['temperature_2m']
print("--- 기상 예측 시간 정보 (상위 5건) ---")
print(hourly_time[:5])
print("\n--- 기상 예측 온도 정보 (상위 5건) ---")
print(hourly_temp[:5])
```

예제 6: API 응답 결과를 Pandas 데이터프레임으로 직접 변환하기

개념: 추출해 낸 시간 리스트와 온도 리스트를 하나의 판다스 데이터프레임 표 형태로 깔끔하게 적재합니다.

프롬프트 입력해보기:

"기상 예측 API에서 추출한 시간 리스트(hourly_time)와 온도 리스트(hourly_temp)를 Pandas 데이터프레임으로 변환해서 '예측시간'과 '온도(°C)' 열을 가지는 표 형태로 깔끔하게 적재하고 상위 10개 행을 보여주는 코드를 작성해줘."

AI가 짜준 코드 실행해보기:

```
import pandas as pd
# 딕셔너리로 정렬한 후 데이터프레임으로 주입
df_weather = pd.DataFrame({
    '예측시간': hourly_time,
    '온도(°C)': hourly_temp
})
print("--- OpenAPI 결과 데이터프레임 화 ---")
print(df_weather.head(10))
```

예제 7: 인증이 필요한 공공데이터포털(data.go.kr) OpenAPI 가상 호출

개념: 공공데이터포털에서 가장 빈번히 쓰이는 '서비스키(ServiceKey)' 인증 헤더와 디코딩 파라미터 적용 방식의 표준 구조를 이해합니다.

프롬프트 입력해보기:

"공공데이터포털(data.go.kr) OpenAPI를 가상 호출하기 위해 파라미터를 딕셔너리로 조립해줘. 파라미터에는 serviceKey(인증키), pageNo(페이지번호), numOfRows(행 개수), dataType('JSON'), base_date('20260714'), nx('55'), ny('127') 등이 포함되어야 해. 템플릿 형태로 작성해줘."

AI가 짜준 코드 실행해보기:

```
# 실제 공공데이터포털 호출 시 활용되는 문법 규격 예시 (시뮬레이션)
api_url = "http://apis.data.go.kr/1360000/VilageFcstInfoService_2.0/getUltraSrtNcst"
service_key = "DECODED_SERVICE_KEY_PLACEHOLDER" # 포털에서 발급받은 디코딩 키
params = {
    'serviceKey': service_key,
    'pageNo': '1',
    'numOfRows': '10',
    'dataType': 'JSON',
    'base_date': '20260714',
    'base_time': '0600',
    'nx': '55',
    'ny': '127'
}
# requests.get(api_url, params=params) 형태로 가동합니다.
print("공공데이터포털 OpenAPI 규격 매핑 파라미터 정의 성공!")
```


예제 8: 대용량 API 수집을 위한 반복문 기반 페이지네이션(Pagination) 처리

개념: 데이터 양이 방대해 여러 페이지로 쪼개져 있는 API의 경우, 페이지 번호(pageNo)를 루프로 돌려 일괄 수집합니다.

프롬프트 입력해보기:

"API 데이터가 페이지로 나뉘어 있을 때 반복문(for)을 사용해 1페이지부터 3페이지까지 일괄 수집하는 코드를 작성해줘. 호출할 주소는 'https://jsonplaceholder.typicode.com/comments' 이고, 각 페이지당 postId=1, _limit=3개의 데이터만 받아서 하나의 리스트에 모아줘."

AI가 짜준 코드 실행해보기:

```
import requests
# 페이지 번호를 바꿔가며 3페이지까지 수집해 통합하는 시뮬레이션
combined_list = []
for page in range(1, 4):
    url = f"https://jsonplaceholder.typicode.com/comments"
    params = {
        'postId': 1,
        '_page': page,
        '_limit': 3 # 페이지당 3건씩 제한
    }
    res = requests.get(url, params=params)
    if res.status_code == 200:
        combined_list.extend(res.json())
print(f"일괄 수집 완료된 댓글 개수: {len(combined_list)}개")
```

예제 9: XML 포맷의 API 응답 데이터 파싱 및 가공 맛보기

개념: 오래된 공공 API에서 여전히 많이 리턴해 주는 XML 구조를 판다스의 read_xml을 통해 단 한 줄로 데이터프레임으로 변환합니다.

프롬프트 입력해보기:

"수자원 정량 데이터가 들어있는 가상의 XML 문자열에서 엘리먼트 내부에 있는 과 정보만 Pandas의 read_xml 함수를 사용해 단 한 줄로 파싱해서 데이터프레임 표로 만드는 코드를 작성해줘."

AI가 짜준 코드 실행해보기:

```
import pandas as pd
# 가상 XML 데이터
xml_data = """
<root>
  <item><name>팔당정수장</name><value>0.12</value></item>
  <item><name>부평정수장</name><value>0.28</value></item>
</root>
"""
# 판다스의 read_xml을 이용해 item 구조의 엘리먼트 추출
df_xml = pd.read_xml(xml_data, xpath='./item')
print("--- XML 파싱 결과 데이터프레임 ---")
print(df_xml)
```

예제 10: 수집한 API 실시간 데이터를 로컬 CSV 파일로 저장하기

개념: 예제 6에서 파싱 완료한 실시간 기상 예측 데이터를 로컬에 엑셀 및 텍스트 파일로 안전하게 추출하여 백업합니다.

프롬프트 입력해보기:

"수집 및 가공이 끝난 기상 예측 데이터프레임(df_weather)을 한글 깨짐 없이 'realtime_weather.csv' 파일로 저장하는 코드를 작성해줘. 인코딩은 'utf-8-sig'로 설정해줘."

AI가 짜준 코드 실행해보기:

```
# 한글 깨짐 방지를 위한 utf-8-sig 인코딩 선언
df_weather.to_csv("realtime_weather.csv", index=False, encoding='utf-8-sig')
print("realtime_weather.csv 백업 완료!")
```

2. Pandas 데이터 가공 기초 및 상관도 분석

예제 11: 딕셔너리를 활용한 수질 계측 데이터프레임 생성

개념: 판다스의 핵심 구조인 2차원 표 형태의 데이터프레임(DataFrame)을 파이썬 딕셔너리로 직접 생성해 봅니다.

프롬프트 입력해보기:

"수계(한강, 낙동강 등), 수온, 탁도, pH, 강수량 정보를 담고 있는 수자원 가상의 수질 데이터프레임(df)을 파이썬 딕셔너리를 이용해 판다스로 생성하는 코드를 작성해 줘."

AI가 짜준 코드 실행해보기:

```
import pandas as pd
# 수자원공사 가상의 수질 데이터 정의 (딕셔너리 구조)
data = {
    '수계': ['한강', '낙동강', '금강', '영산강', '한강', '낙동강'],
    '수온(°C)': [15.2, 18.5, 16.1, 19.8, 14.8, 17.9],
    '탁도(NTU)': [0.35, 1.25, 0.42, 1.85, 0.28, 1.10],
    'pH': [7.2, 8.1, 7.5, 6.2, 7.4, 8.3],
    '강수량(mm)': [10.0, 120.0, 15.0, 180.0, 5.0, 95.0]
}
# 데이터프레임 생성
df = pd.DataFrame(data)
print(df)
```

예제 12: 데이터프레임 상위/하위 행 및 차원 확인 (head, tail, shape)

개념: 대용량 데이터를 열었을 때 데이터의 일부를 엿보거나 전체 행/열의 개수를 파악하는 기본 도구입니다.

프롬프트 입력해보기:

"생성한 데이터프레임에서 상위 3개 행을 조회해 보여주고, 데이터프레임의 전체 크기(행과 열 개수)를 튜플 형태로 리턴해 확인하는 코드를 작성해줘."

AI가 짜준 코드 실행해보기:

```
# 상위 3개 행 출력
print("--- 상위 3개 행 ---")
print(df.head(3))
# 데이터프레임의 크기 (행, 열 개수)
print("\n--- 데이터프레임 크기 (행, 열) ---")
print(df.shape)
```

예제 13: 데이터 구조 정보 및 기본 기술통계 확인 (info, describe)

개념: 컬럼들의 데이터 타입(실수형, 문자형 등)과 빈 칸(결측치) 여부, 그리고 평균, 최소/최대값 등 통계 요약 정보를 한눈에 봅니다.

프롬프트 입력해보기:

"데이터프레임의 요약 정보(info)를 확인하고, 수치형 컬럼들의 평균, 최대/최소, 사분위수 통계 수치들을 요약(describe)해서 인쇄해주는 코드를 작성해 줘."

AI가 짜준 코드 실행해보기:

```
print("--- 데이터 정보 ---")
df.info()
print("\n--- 기술 통계 요약 ---")
print(df.describe())
```

예제 14: 특정 열(Column) 및 특정 행 인덱싱 기법

개념: 표에서 내가 원하는 컬럼(열)이나 로우(행)만 콧 집어서 꺼내와서 확인합니다.

프롬프트 입력해보기:

"데이터프레임에서 '수계'와 '탁도(NTU)' 열만 가져와서 선택하고, 추가로 2번째 행부터 4번째 행까지 범위를 선택해서 출력해주는 코드를 작성해줘."

AI가 짜준 코드 실행해보기:

```
# '수계'와 '탁도(NTU)' 열만 선택
print("--- 특정 열 선택 ---")
print(df[['수계', '탁도(NTU)']])
# 2번째 행(인덱스 1)부터 4번째 행(인덱스 3)까지 슬라이싱 선택
print("\n--- 특정 행 범위 선택 ---")
print(df.iloc[1:4])
```

예제 15: 단일 조건 필터링 (탁도 0.5 NTU 초과 지점 검출)

개념: 정수장 운영 기준치인 탁도 0.5 NTU를 초과하는 비정상 관리 구역 데이터를 조건식 필터링으로 걸러냅니다.

프롬프트 입력해보기:

"데이터프레임에서 '탁도(NTU)' 열의 수치가 0.5 NTU를 초과하는 위험 관측 지점들의 행만 필터링해서 뽑아내는 코드를 작성해줘."

AI가 짜준 코드 실행해보기:

```
# 탁도가 0.5 NTU를 초과하는 행만 추출
turbid_danger = df[df['탁도(NTU)'] > 0.5]
print("--- 탁도 0.5 초과 위험 관측점 ---")
print(turbid_danger)
```


예제 16: 복합 조건 필터링 (pH가 정상 범위를 벗어난 이상치 검출)

개념: pH가 6.5보다 낮거나 또는 8.5보다 높은 '비정상 산성/알칼리성' 지점을 | (OR) 연산자로 필터링합니다.

프롬프트 입력해보기:

"수질 데이터프레임에서 pH가 6.5 미만이거나 8.5를 초과하는 비정상적인 수질을 나타내는 행들을 필터링해 확인하는 복합 조건문 코드를 작성해줘."

AI가 짜준 코드 실행해보기:

```
# pH 정상범위(6.5 ~ 8.5)를 벗어난 경우 필터링 (OR 조건 사용)
ph_outlier = df[(df['pH'] < 6.5) | (df['pH'] > 8.5)]
print("--- pH 관리 기준 이탈 지역 ---")
print(ph_outlier)
```

예제 17: 새로운 파생 변수(열) 생성 및 수식 연산

개념: 기존 데이터를 가공하여 사칙연산을 수행한 결과를 새로운 컬럼으로 표에 추가합니다.

프롬프트 입력해보기:

"수온(°C) 컬럼을 활용해 '수온(°F)' 화씨 온도 컬럼을 새로 만들어줘. 공식은 (섭씨온도 * 9/5) + 32 이며, 이를 적용해 데이터프레임에 추가하고 출력하는 코드를 만들어줘."

AI가 짜준 코드 실행해보기:

```
# 수온을 화씨(°F) 온도로 변환하여 파생변수 생성
df['수온(°F)'] = (df['수온(°C)'] * 9/5) + 32
print(df)
```

예제 18: 그룹화 분석 (groupby를 활용한 수계별 평균 수온 및 탁도 연산)

개념: 엑셀의 피벗 테이블처럼 데이터를 특정 그룹(예: 수계별)으로 묶어 각 그룹별 평균을 구합니다.

프롬프트 입력해보기:

"데이터프레임의 데이터를 '수계'별로 그룹화(groupby)해서, 수계 그룹별 수온, 탁도, pH, 강수량의 평균값을 표 형태로 요약해 계산하는 코드를 작성해 줘."

AI가 짜준 코드 실행해보기:

```
# 수계 그룹별 평균값 구하기 (수치형 열만 대상)
numeric_cols = ['수온(°C)', '탁도(NTU)', 'pH', '강수량(mm)']
groupby_result = df.groupby('수계')[numeric_cols].mean()
print("--- 수계별 수질 항목 평균치 ---")
print(groupby_result)
```

예제 19: 수질 항목 간 상관도 분석 (corr() 함수를 이용한 상관계수 구하기)

개념: 수온, 탁도, pH, 강수량 등 수질 인자들이 서로 어떤 밀접한 관계를 맺고 있는지 -1에서 1 사이의 수치(상관계수)로 계산합니다.

프롬프트 입력해보기:

"수온, 탁도, pH, 강수량 컬럼 간의 피어슨 상관계수 행렬을 corr() 함수로 구해서 인쇄해주는 파이썬 코드를 작성해줘."

AI가 짜준 코드 실행해보기:

```
# 수치형 변수들 간의 피어슨 상관계수 행렬 구하기
correlation_matrix = df[numeric_cols].corr()
print("--- 수질 인자 간 상관계수 행렬 ---")
print(correlation_matrix)
```

예제 20: 결측값 탐지 및 대체 (isnull, fillna)

개념: 센서 오류나 정전 등으로 유실되어 비어있는 데이터(NaN)를 찾아내고, 이를 평균값으로 안전하게 채워넣는 기법입니다.

프롬프트 입력해보기:

"데이터프레임 내부에 강제 결측치(NaN)를 하나 심고, 결측치가 있는지 검출한 뒤(isnull), 비어있는 행의 탁도 값을 전체 탁도의 평균값으로 안전하게 메꾸는(fillna) 코드를 작성해줘."

AI가 짜준 코드 실행해보기:

```
import numpy as np
# 강제로 빈 칸(결측치)이 포함된 임시 데이터프레임 생성
df_nan = df.copy()
df_nan.loc[2, '탁도(NTU)'] = np.nan # 2번 행의 탁도를 비움
print("--- 빈 칸 존재 여부 확인 (True가 빈 칸) ---")
print(df_nan['탁도(NTU)'].isnull())
# 비어있는 탁도 칸을 전체 탁도 평균값으로 대체
mean_turbidity = df_nan['탁도(NTU)'].mean()
df_clean = df_nan.fillna({'탁도(NTU)': mean_turbidity})
print("\n--- 빈 칸을 평균값으로 대체한 결과 ---")
print(df_clean)
```

3. Matplotlib & Seaborn 데이터 시각화 및 상관도 표현

예제 21: 차트 한글 폰트 설정 및 음수 기호 깨짐 방지

개념: 파이썬 차트 라이브러리는 기본 영문 지원이므로, 한글이 네모 박스로 깨져 나오는 현상을 맑은 고딕(Malgun Gothic)으로 방지하고 음수 부호를 올바르게 노출시킵니다.

프롬프트 입력해보기:

"Matplotlib 차트에 한글이 깨지지 않고 마이너스 부호가 표시되도록 맑은 고딕('Malgun Gothic') 폰트 설정 및 유니코드 마이너스 예외 처리를 반영하는 코드를 작성해줘."

AI가 짜준 코드 실행해보기:

```
import matplotlib.pyplot as plt
import seaborn as sns
# 한글 깨짐 예방 설정
plt.rcParams['font.family'] = 'Malgun Gothic'
plt.rcParams['axes.unicode_minus'] = False
print("한글 폰트 및 마이너스 부호 설정 완료!")
```

예제 22: 일자별 수온 추이를 나타내는 꺾은선 그래프 그리기 (plt.plot)

개념: 시간에 따른 데이터의 흐름과 변화 추세를 파악하기 위해 마커가 표시된 꺾은선 그래프를 그립니다.

프롬프트 입력해보기:

"일자별 수온 변화 데이터를 꺾은선 그래프로 시각화해줘. 선 색상은 '#0080ff', 모양은 점(marker='o'), 점선 스타일로 그리고 제목과 축 레이블을 한글로 달고 그리드 눈금도 추가해줘."

AI가 짜준 코드 실행해보기:

```
# 가상 시계열 데이터
dates = ['07-01', '07-02', '07-03', '07-04', '07-05', '07-06']
temp = [18.2, 18.9, 19.5, 17.8, 16.5, 17.2]
plt.figure(figsize=(7, 4))
plt.plot(dates, temp, color='#0080ff', marker='o', linestyle='--', linewidth=2)
plt.title("일자별 원수 수온 변화 추이")
plt.xlabel("측정 일자")
plt.ylabel("수온 (°C)")
plt.grid(True, linestyle=':', alpha=0.6)
plt.show()
```

예제 23: 정수장별 평균 탁도 비교 막대그래프 그리기 (sns.barplot)

개념: 각 정수장(카테고리)별로 탁도 수치의 크기 편차를 직관적으로 비교할 수 있는 막대 차트를 그립니다.

프롬프트 입력해보기:

"정수장 4곳의 평균 탁도 비교 데이터를 가로축에 정수장 명칭, 세로축에 평균 탁도로 설정해 Seaborn의 barplot 막대그래프로 그리는 코드를 작성해줘.
색상 스타일은 파란색 계열(palette='Blues_r')로 설정해줘."

AI가 짜준 코드 실행해보기:

```
plants = ['팔당정수장', '부평정수장', '덕소정수장', '성남정수장']
avg_turbidity = [0.15, 0.42, 0.21, 0.38]
plt.figure(figsize=(7, 4))
sns.barplot(x=plants, y=avg_turbidity, palette='Blues_r')
plt.title("정수장별 평균 탁도 비교")
plt.xlabel("정수장 명칭")
plt.ylabel("평균 탁도 (NTU)")
plt.show()
```


예제 24: 수계별 데이터 분포 비교를 위한 박스플롯 (sns.boxplot)

개념: 데이터의 최댓값, 최솟값, 중앙값 및 이상치(Outlier)의 분포를 한눈에 박스 모형으로 조망합니다.

프롬프트 입력해보기:

"한강과 낙동강 수계의 pH 값 분포 편차를 비교해보고 싶어. X축을 수계, Y축을 pH로 두고 Seaborn의 boxplot으로 데이터 분포를 보여주는 코드를 작성해줘."

AI가 짜준 코드 실행해보기:

```
# 한강과 낙동강의 다수 계측 데이터 가상화
box_data = {
    '수계': ['한강']*20 + ['낙동강']*20,
    'pH': [7.1, 7.2, 7.3, 7.4, 7.2, 7.5, 7.3, 7.2, 7.1, 7.4,
           7.2, 7.3, 8.8, 7.2, 7.1, 7.3, 7.4, 7.2, 7.1, 7.3,
           8.1, 8.2, 8.3, 8.0, 8.4, 8.2, 8.1, 8.3, 8.2, 8.5,
           8.0, 8.1, 8.3, 8.4, 8.2, 8.1, 6.2, 8.3, 8.2, 8.1]
}
df_box = pd.DataFrame(box_data)
plt.figure(figsize=(7, 4))
sns.boxplot(x='수계', y='pH', data=df_box, palette='Set2')
plt.title("수계별 pH 데이터 분포 및 이상치 확인")
plt.show()
```

예제 25: 강수량과 탁도의 상관관계 분석을 위한 산점도 및 회귀선 그리기 (sns.scatterplot, sns.regplot)

개념: 두 연속형 수치 데이터 간의 상관관계를 점들의 분포 및 1차 회귀 직선으로 선명하게 표출합니다.

프롬프트 입력해보기:

"강수량과 탁도의 가상 상관 데이터프레임을 생성하고, X축을 강수량, Y축을 탁도로 두어 Seaborn의 regplot으로 산점도 위에 1차 회귀 추세선까지 겹쳐 그리는 상관관계 시각화 코드를 작성해줘."

AI가 짜준 코드 실행해보기:

```
# 강수량과 탁도 데이터
rainfall_data = [10.0, 30.0, 50.0, 80.0, 120.0, 150.0, 20.0, 90.0, 110.0, 140.0]
turbidity_data = [0.15, 0.25, 0.45, 0.60, 1.20, 1.85, 0.18, 0.75, 1.10, 1.60]
df_corr = pd.DataFrame({'강수량(mm)': rainfall_data, '탁도(NTU)': turbidity_data})
plt.figure(figsize=(7, 4))
sns.regplot(x='강수량(mm)', y='탁도(NTU)', data=df_corr, color='red', marker='x')
plt.title("강수량과 탁도의 상관관계 분석 (산점도+회귀선)")
plt.show()
```

예제 26: 수질 인자 간 상관계수 행렬을 색상으로 시각화하는 상관 히트맵 (sns.heatmap)

개념: 2교시에서 계산한 상관계수 행렬을 직관적인 컬러 맵 격자로 출력하여 다변수 상관도를 한눈에 드러냅니다.

프롬프트 입력해보기:

"수온, 탁도, pH, 강수량 등 다변수 수질 데이터의 상관행렬(corr())을 계산하고, 그 상관계수 수치를 격자 칸에 텍스트로 박아주며(annot=True) 색채 맵(cmap='RdBu')으로 표현하는 Seaborn 히트맵(heatmap) 차트 코드를 작성해줘."

AI가 짜준 코드 실행해보기:

```
# 가상 수질 상관 데이터프레임의 상관계수 계산
df_matrix = pd.DataFrame({
    '수온': [12, 14, 18, 22, 25, 26],
    '탁도': [0.1, 0.2, 0.5, 0.9, 1.4, 1.7],
    'pH': [7.2, 7.3, 7.5, 7.8, 8.1, 8.2],
    '강수량': [0, 10, 45, 90, 130, 160]
})
corr = df_matrix.corr()
plt.figure(figsize=(6, 5))
# annot=True 옵션으로 격자 안에 상관계수 수치를 인쇄함
sns.heatmap(corr, annot=True, cmap='RdBu', vmin=-1, vmax=1, linewidths=0.5)
plt.title("수질 인자 간 상관관계 Heatmap")
plt.show()
```

예제 27: 정수장별 공급 비율 시각화 원형 차트 (plt.pie)

개념: 전체 용수 공급량 중에서 각 정수장이 차지하는 기여 백분율 비율을 둥근 파이 형태로 표현합니다.

프롬프트 입력해보기:

"한강, 낙동강, 금강, 영산강 정수장의 용수 공급 비율 데이터(45, 30, 15, 10)를 준비하고, 각 파이 조각의 백분율이 표시되며 한강 부분이 약간 빠져나온 (explode) 원형 차트(pie)를 그리는 코드를 작성해줘."

AI가 짜준 코드 실행해보기:

```
labels = ['한강정수장', '낙동강정수장', '금강정수장', '영산강정수장']
shares = [45, 30, 15, 10]
colors = ['#ff9999', '#66b3ff', '#99ff99', '#ffcc99']
plt.figure(figsize=(5, 5))
plt.pie(shares, labels=labels, autopct='%1.1f%%', startangle=90,
        colors=colors, explode=(0.05, 0, 0, 0))
plt.title("수계별 용수 공급 점유 비율")
plt.show()
```

예제 28: 타이틀, 범례, 축 이름, 눈금선(Grid) 추가 및 꾸미기

개념: 제3자가 차트를 해석하기 쉽도록 한글 캡션(제목, 축 눈금, 라벨)과 범례 안내판을 적절히 도식합니다.

프롬프트 입력해보기:

"A정수장과 B정수장의 일자별 탁도 추이를 꺾은선으로 겹쳐 그리되, 각각의 선에 범례(legend) 이름표를 달아주고 차트 제목, 축 제목을 적어주며, 눈금선(Grid) 까지 추가해서 스타일을 꾸미는 코드를 작성해줘."

AI가 짜준 코드 실행해보기:

```
days = ['1일', '2일', '3일', '4일', '5일']
turb1 = [0.12, 0.18, 0.45, 0.23, 0.11]
turb2 = [0.08, 0.35, 0.92, 0.41, 0.15]
plt.figure(figsize=(7, 4))
plt.plot(days, turb1, label='A 정수장', color='green', marker='o')
plt.plot(days, turb2, label='B 정수장', color='orange', marker='s')
plt.title("일자별 정수장 수계 탁도 비교 관측", fontsize=14, fontweight='bold')
plt.xlabel("측정 날짜")
plt.ylabel("탁도 수치 (NTU)")
plt.legend(loc='upper right') # 우측 상단 범례 노출
plt.grid(True, which='both', axis='both', color='gray', linestyle='--', alpha=0.3)
plt.show()
```

예제 29: 한 화면에 2개 이상의 그래프 분할 배치하기 (plt.subplots)

개념: 하나의 이미지 파일 액자(Figure) 안에 여러 장의 도화지(Axes)를 배치하여, 두 가지 다른 차트를 한눈에 대조합니다.

프롬프트 입력해보기:

"Matplotlib subplots를 써서 가로로 1행 2열 분할 도화지를 준비해줘. 왼쪽 도화지(axes[0])에는 수온 선 그래프를 그리고, 오른쪽 도화지(axes[1])에는 pH 막대 그래프를 개별적으로 그려 한 화면에 띄우는 코드를 작성해줘."

AI가 짜준 코드 실행해보기:

```
fig, axes = plt.subplots(1, 2, figsize=(10, 4)) # 1행 2열의 도화지 분할
# 왼쪽 그래프: 수온 추이
axes[0].plot(['1일', '2일', '3일'], [15.5, 16.2, 17.1], color='red', marker='^')
axes[0].set_title("일자별 수온")
axes[0].set_ylabel("온도 (°C)")
# 오른쪽 그래프: pH 추이
axes[1].bar(['1일', '2일', '3일'], [7.2, 7.5, 7.1], color='purple')
axes[1].set_title("일자별 pH")
axes[1].set_ylabel("pH 수치")
plt.tight_layout() # 간격 자동 조절
plt.show()
```

예제 30: 강수량(막대)과 탁도(선)를 한데 그리는 이중 Y축 차트 (twinx)

개념: 두 수치의 단위(mm vs NTU)가 너무 달라 하나의 Y축으로는 비교하기 불가능할 때, 왼쪽과 오른쪽 Y축을 다르게 설정해 포깁니다.

프롬프트 입력해보기:

"강수량(mm)과 탁도(NTU)는 단위가 달라서 한 Y축에 그리면 탁도 선이 찌그러져. 왼쪽 Y축은 강수량 막대그래프를 그리고, twinx()를 써서 배경을 공유하는 오른쪽 Y축에 탁도 선그래프를 겹쳐 그리는 이중 Y축 차트 코드를 만들어줘."

AI가 짜준 코드 실행해보기:

```
days = ['월', '화', '수', '목', '금']
rainfall = [0, 45, 130, 20, 5]
turbidity = [0.12, 0.32, 1.45, 0.85, 0.30]
fig, ax1 = plt.subplots(figsize=(8, 4))
# 첫 번째 Y축 (왼쪽): 강수량 막대 그래프
ax1.bar(days, rainfall, color='#9cd3ff', label='강수량', alpha=0.7)
ax1.set_xlabel("요일")
ax1.set_ylabel("강수량 (mm)", color='blue')
ax1.tick_params(axis='y', labelcolor='blue')
# 두 번째 Y축 (오른쪽): 탁도 꺾은선 그래프
ax2 = ax1.twinx()
ax2.plot(days, turbidity, color='red', marker='o', linewidth=2.5, label='탁도')
ax2.set_ylabel("탁도 (NTU)", color='red')
ax2.tick_params(axis='y', labelcolor='red')
plt.title("강수량 변화에 따른 탁도 상관성 분석 (이중축)")
plt.show()
```

4. SQLite 데이터베이스 기초 및 외부 연동

예제 31: SQLite3 DB 접속 및 연결 커넥션 생성

개념: 파이썬 기본 탑재 모듈인 sqlite3를 열어 로컬에 파일 창고(.db)를 생성하고 연결 통로(Connection)를 엽니다.

프롬프트 입력해보기:

"파이썬 sqlite3 라이브러리를 사용해서 'kwater_db.db' 파일 데이터베이스를 신규 생성 및 연결하고, 연결 성공 메시지를 띄운 뒤 커넥션을 안전하게 닫는(close) 코드를 작성해줘."

AI가 짜준 코드 실행해보기:

```
import sqlite3
# kwater_db.db 파일에 접속 (파일이 없으면 자동 새 생성)
conn = sqlite3.connect("kwater_db.db")
print("데이터베이스 연결 성공!")
# 작업 종료 후 통로 닫기
conn.close()
```


예제 32: 수질 이력 저장용 테이블 정의 (CREATE TABLE)

개념: DB 창고 내부 안에 엑셀 시트 형태의 격자 틀을 정의하는 테이블을 설계합니다.

프롬프트 입력해보기:

"sqlite3 데이터베이스 테이블을 만들어볼게. 테이블 이름은 TB_WATER_LOG이고 컬럼은 id(자동증가 기본키), river_name(문자열), temperature(실수), turbidity(실수), rainfall(실수)을 포함하도록 SQL문을 작성해서 실행 및 커밋하는 코드를 만들어줘."

AI가 짜준 코드 실행해보기:

```
import sqlite3
conn = sqlite3.connect("kwater_db.db")
cursor = conn.cursor() # DB 제어용 로봇 팔(Cursor) 생성
# 수계, 수온, 탁도, 강수량 항목을 가지는 테이블 생성 SQL 실행
cursor.execute("""
    CREATE TABLE IF NOT EXISTS TB_WATER_LOG (
        id INTEGER PRIMARY KEY AUTOINCREMENT,
        river_name TEXT,
        temperature REAL,
        turbidity REAL,
        rainfall REAL
    )
""")
conn.commit() # 결재 도장 (영구 반영)
conn.close()
print("테이블 생성 완료!")
```

예제 33: 새로운 수질 레코드 단건 삽입 (INSERT INTO)

개념: 물류 데이터를 하나씩 행 단위로 DB 테이블 안에 주입하여 저장합니다.

프롬프트 입력해보기:

"생성한 TB_WATER_LOG 테이블에 강 수계 데이터 1행(river_name='한강', temperature=16.5, turbidity=0.28, rainfall=12.0)을 삽입(INSERT INTO)하고 변경사항을 영구 보존하기 위해 커밋하는 코드를 작성해줘."

AI가 짜준 코드 실행해보기:

```
import sqlite3
conn = sqlite3.connect("kwater_db.db")
cursor = conn.cursor()
# 데이터 1행 추가
cursor.execute("""
    INSERT INTO TB_WATER_LOG (river_name, temperature, turbidity, rainfall)
    VALUES ('한강', 16.5, 0.28, 12.0)
""")
conn.commit()
conn.close()
print("데이터 단건 추가 완료!")
```

예제 34: 튜플 리스트를 활용한 다중 레코드 일괄 삽입 (executemany)

개념: 리스트 형태의 묶음 데이터를 한 번에 밀어 넣어 DB 주입 효율을 극대화합니다.

프롬프트 입력해보기:

"낙동강, 금강, 영산강 등의 수질 데이터를 튜플 리스트 형태의 다중 데이터로 정의하고, sqlite3의 executemany 함수를 사용해 물음표(?) 치환 홀더에 매핑하여 테이블에 일괄 대량 주입하는 코드를 작성해줘."

AI가 짜준 코드 실행해보기:

```
import sqlite3
conn = sqlite3.connect("kwater_db.db")
cursor = conn.cursor()
# 일괄 삽입할 튜플 리스트 데이터 정의
batch_data = [
    ('낙동강', 19.2, 1.15, 140.0),
    ('금강', 17.0, 0.38, 25.0),
    ('영산강', 20.5, 1.95, 210.0),
    ('한강', 15.0, 0.22, 5.0)
]
# ? 기호 자리에 순서대로 튜플 데이터 매핑 주입
cursor.executemany("""
    INSERT INTO TB_WATER_LOG (river_name, temperature, turbidity, rainfall)
    VALUES (?, ?, ?, ?)
""", batch_data)
conn.commit()
conn.close()
print("다중 데이터 일괄 삽입 성공!")
```

예제 35: 전체 데이터 조회 및 출력 (SELECT)

개념: DB에 저장되어 영구 보존된 데이터 행들을 다시 파이썬 메모리로 끄집어냅니다.

프롬프트 입력해보기:

"데이터베이스 TB_WATER_LOG 테이블의 모든 데이터를 조회(SELECT * FROM)하여 로봇 팔(cursor.fetchall())로 한 줄씩 꺼낸 후 화면에 반복문으로 인쇄 출력해주는 코드를 작성해줘."

AI가 짜준 코드 실행해보기:

```
import sqlite3
conn = sqlite3.connect("kwater_db.db")
cursor = conn.cursor()
# 모든 컬럼 조회
cursor.execute("SELECT * FROM TB_WATER_LOG")
rows = cursor.fetchall() # 조회된 행들의 리스트 전체 반환
print("--- DB 저장 데이터 목록 ---")
for r in rows:
    print(r)
conn.close()
```

예제 36: 특정 지역의 수질 데이터만 필터링 조회 (WHERE 조건절)

개념: SQL 쿼리에 특정 조건(WHERE)을 부여하여 '한강' 수계만 선별 조회합니다.

프롬프트 입력해보기:

"데이터베이스에서 강 이름이 '한강'인 데이터 행들만 필터링 조회(WHERE river_name = ?)해서 화면에 뽑아내 주는 SQL 실행 코드를 작성해줘."

AI가 짜준 코드 실행해보기:

```
import sqlite3
conn = sqlite3.connect("kwater_db.db")
cursor = conn.cursor()
# river_name이 '한강'인 행만 조회
cursor.execute("SELECT * FROM TB_WATER_LOG WHERE river_name = ?", ('한강',))
hangang_rows = cursor.fetchall()
print("--- 한강 수계 계측 이력 ---")
for r in hangang_rows:
    print(r)
conn.close()
```

예제 37: 입력 오류가 있는 데이터 수정 반영 (UPDATE)

개념: 이미 들어간 기존 행의 수치 오류를 고쳐 올바르게 갱신합니다.

프롬프트 입력해보기:

"데이터베이스에 잘못 입력된 수치를 고쳐볼게. 테이블에서 id가 1번인 행의 temperature(수온) 값을 15.0으로 교체 및 수정(UPDATE)하고 영구 반영하는 코드를 작성해줘."

AI가 짜준 코드 실행해보기:

```
import sqlite3
conn = sqlite3.connect("kwater_db.db")
cursor = conn.cursor()
# ID가 1번인 계측점의 수온을 15.0으로 갱신
cursor.execute("UPDATE TB_WATER_LOG SET temperature = ? WHERE id = ?", (15.0, 1))
conn.commit()
conn.close()
print("데이터 수정 완료!")
```

예제 38: 필요 없는 데이터 삭제 (DELETE)

개념: 중복되거나 비정상 계층값인 데이터 행을 삭제하여 DB 용량과 정합성을 정제합니다.

프롬프트 입력해보기:

"데이터베이스 테이블에서 id가 2번인 이상치 데이터 행 하나를 완전히 테이블에서 제거 및 삭제(DELETE) 처리하는 코드를 작성해줘."

AI가 짜준 코드 실행해보기:

```
import sqlite3
conn = sqlite3.connect("kwater_db.db")
cursor = conn.cursor()
# ID가 2번인 데이터를 테이블에서 제거
cursor.execute("DELETE FROM TB_WATER_LOG WHERE id = ?", (2,))
conn.commit()
conn.close()
print("데이터 삭제 완료!")
```



예제 39: SQL 쿼리 결과를 판다스 데이터프레임으로 직접 변환 (pd.read_sql)

개념: 굳이 fetchall()을 돌지 않고, 판다스 전용 함수로 SQL 쿼리 전체를 즉시 표 구조로 변환합니다.

프롬프트 입력해보기:

"데이터베이스 kwater_db.db에 연결한 후, Pandas의 read_sql 함수를 사용해 TB_WATER_LOG의 모든 데이터를 단 한 줄의 코드로 판다스 데이터프레임 (df_db) 표로 끌고 와서 보여주는 코드를 작성해줘."

AI가 짜준 코드 실행해보기:

```
import sqlite3
import pandas as pd
conn = sqlite3.connect("kwater_db.db")
# 단 한 줄로 DB 테이블 데이터를 데이터프레임으로 주입
df_db = pd.read_sql("SELECT * FROM TB_WATER_LOG", conn)
conn.close()
print("--- DB에서 로드된 판다스 데이터프레임 ---")
print(df_db)
```



예제 40: 판다스에서 정제 완료된 표를 DB 테이블로 일괄 저장 (to_sql)

개념: 판다스에서 가공과 연산을 거친 데이터프레임 표 자체를 DB에 테이블을 자동 생성하며 그대로 저장해 버립니다.

프롬프트 입력해보기:

"새로운 강 이름, 수온, 탁도, 강수량을 가진 Pandas 데이터프레임(df_new)을 수동 생성하고, 이 데이터프레임 표 자체를 sqlite3 커넥션을 활용해 'TB_WATER_CLEAN' 이라는 신규 DB 테이블로 통째로 저장(to_sql) 백업하는 코드를 작성해줘."

AI가 짜준 코드 실행해보기:

```
import sqlite3
import pandas as pd
conn = sqlite3.connect("kwater_db.db")
# 새로운 정제 수질 데이터프레임 준비
df_new = pd.DataFrame({
    'river_name': ['영산강', '금강'],
    'temperature': [21.0, 18.0],
    'turbidity': [2.20, 0.40],
    'rainfall': [250.0, 30.0]
})
# TB_WATER_CLEAN 이라는 신규 테이블로 데이터프레임을 주입 저장 (이미 있으면 덮어쓰기)
df_new.to_sql("TB_WATER_CLEAN", conn, if_exists="replace", index=False)
conn.close()
print("데이터프레임의 DB 테이블 백업 저장 성공!")
```

[프로젝트 1] CSV 수질 데이터 로더 및 통계 조회 GUI 앱

상황 부여: 텍스트 형태의 콘솔 프로그램은 현장 작업자가 사용하기 불편합니다. 윈도우 파일 탐색기를 통해 수질 측정 CSV 파일을 선택하고, 그 안의 데이터를 표(그리드) 형태로 조회하며 하단에 자동으로 수온/탁도 등 요약 통계를 실시간으로 출력해 주는 프로그램 창을 빌드업합니다.

1단계: 실실용 CSV 데이터 생성 및 기본 창 레이아웃 잡기

프롬프트 입력해보기:

"PyQt6를 사용하여 맨 위에 'CSV 파일 불러오기' 버튼을 두고, 중앙에는 빈 표(QTableWidget)를 배치한 뒤, 맨 아래에는 요약 통계 결과가 문자형으로 표시될 라벨(QLabel)이 정렬되도록 세로 정렬 레이아웃(QVBoxLayout)을 적용한 데스크톱 윈도우 창 기본 뼈대 코드를 작성해줘."

2단계: 윈도우 탐색기로 CSV 파일 선택 및 테이블 연동

프롬프트 입력해보기:

"이전에 만든 PyQt6 코드의 'CSV 파일 불러오기' 버튼 클릭 시그널에 동작 함수를 연결해줘.

1. 버튼을 누르면 `QFileDialog.getOpenFileName`을 사용해 윈도우 탐색기가 열리게 해줘.
2. 선택한 CSV 파일을 Pandas로 로드(`pd.read_csv`)하게 해줘.
3. 로드된 데이터프레임의 행과 열 개수만큼 `QTableWidget`의 차원을 맞춘 뒤, 각 격자 칸에 데이터를 루프를 돌며 아이템(`QTableWidgetItem`)으로 출력하게 기능을 연결해줘."

3단계 (최종): Pandas 기반 수치 통계 계산 기능 결합

프롬프트 입력해보기:

"이전 코드에 통계 분석 연산을 보강할게.

1. CSV 파일 로드가 성공한 직후, 로드된 데이터프레임에서 숫자 형식을 가지는 열만 판다스로 필터링해줘.
2. 각 수치 열들의 평균과 최댓값을 구해서 문자열 양식으로 조립해줘.
3. 조립한 요약 통계 텍스트를 화면 맨 아래 라벨(QLabel)에 실시간으로 표시하는 최종 프로젝트 1 완성 코드를 작성해줘."

[프로젝트 2] 조건부 데이터 필터링 및 엑셀 내보내기 GUI 앱

상황 부여: 프로젝트 1은 수입된 원 데이터 전체만 보여줍니다. 이번에는 수질 기준 한계 임계값을 마우스나 스펀박스로 입력받아, 특정 수치(예: 탁도 0.5 NTU 초과)를 초과하는 위험 데이터만 발췌해 표를 새로고침하고, 이 필터링된 보고서만 따로 엑셀 파일로 출력하는 기능을 완성합니다.

1단계: 임계치 조절 스펀박스(QDoubleSpinBox) 추가 및 GUI 레이아웃 구성

프롬프트 입력해보기:

"PyQt6를 사용해서 상단 헤더 영역에 'CSV 파일 로드' 버튼, '탁도 기준치 초과 필터:'라는 라벨, 그리고 소수점 입력이 가능하고 기본값이 0.50으로 세팅된 QDoubleSpinBox 컴포넌트와 '필터 적용' 버튼을 차례대로 가로 정렬 배치해줘. 하단에는 표(QTableWidget)와 '필터링된 데이터 엑셀 내보내기' 버튼, 그리고 상태 결과를 알려줄 QLabel 라벨을 세로로 쌓은 기본 GUI 코드를 작성해줘."

2단계: 스피너박스 수치 연동 판다스 필터링 구현

프롬프트 입력해보기:

"이전 GUI 코드에 기능을 추가할게.

1. 'CSV 파일 로드' 버튼에 파일 열기 대화상자를 연동해서 CSV를 열고 표에 출력하게 해줘.
2. '필터 적용' 버튼을 누르면, 로드된 원본 데이터프레임에서 '타도' 컬럼 값이 QDoubleSpinBox의 설정값보다 큰 행들만 판다스 조건식으로 필터링하게 해줘.
3. 필터링된 행들만 표(QTableWidget)에 다시 출력하도록 업데이트 함수를 작성해줘."

3단계 (최종): 테이블 내부 데이터를 엑셀 보고서 파일로 내보내기

프롬프트 입력해보기:

"이전 코드의 '엑셀 내보내기' 버튼에 저장 기능을 연결할게."

1. 버튼을 클릭하면 `QFileDialog.getSaveFileName`을 호출해 사용자가 저장할 엑셀 파일(.xlsx) 경로를 지정받게 해줘.
2. 현재 `QTableWidget` 화면 상에 리스트업되어 보여지고 있는 데이터들만 다시 판다스 데이터프레임으로 역추출해줘.
3. 추출한 데이터프레임을 지정된 파일 경로에 `to_excel` 명령으로 저장하여 엑셀 보고서 사출을 완료하는 최종 완성본 코드를 작성해줘."

[프로젝트 3] 수계별 분석 및 Matplotlib 차트 임베딩 GUI 앱

상황 부여: 수자원 분석 장표의 핵심은 그래프 시각화입니다. 드롭다운 콤보박스에서 특정 수계(한강, 낙동강 등)를 마우스로 클릭해서 바꾸면, 프로그램 화면 우측에 임베딩된 Matplotlib 도화지 영역에 해당 수계의 강수량 대비 탁도 추이 이중축 차트가 즉각 동적으로 다시 그려지는 차트 대시보드를 생성합니다.

1단계: 수계 선택 콤보박스 및 Matplotlib 캔버스 임베딩 UI

프롬프트 입력해보기:

"PyQt6 레이아웃을 좌우로 분할해줘.

1. 왼쪽 영역에는 '수계 선택:' QLabel과 QComboBox, 그리고 '차트 이미지 저장' 버튼을 세로로 쌓아 배치해줘.
2. 오른쪽 영역에는 Matplotlib의 Figure와 Axes를 담은 FigureCanvasQTAgi 캔버스를 이식해줘.
3. 한글 깨짐 방지 맑은고딕 폰트가 차트에 미리 기본 매핑되도록 생성자를 구성한 뼈대 소스코드를 작성해줘."

2단계: 콤보박스 시그널 연결 및 동적 이중축 차트 갱신

프롬프트 입력해보기:

"이전 코드의 콤보박스(QComboBox) 변경 시그널(currentIndexChanged)을 차트 갱신 함수에 연결해줘.

1. 수계 콤보박스에서 항목을 선택하면 해당 수계의 로우들만 판다스 데이터프레임에서 필터링해줘.
2. 기존 Matplotlib Axes 그림을 ax.clear()로 지워내줘.
3. 선택된 데이터를 활용해 왼쪽 Y축에는 강수량 막대그래프를, 오른쪽 Y축(twinx 사용)에는 탁도 꺾은선그래프를 겹쳐 그린 뒤 canvas.draw()로 화면을 즉시 새로고침하는 코드를 작성해줘."

3단계 (최종): 차트 이미지 로컬 파일 저장 기능 결합

프롬프트 입력해보기:

"이전 코드의 '차트 이미지 저장' 버튼에 기능을 심어줘."

1. 버튼 클릭 시 파일 저장 대화상자(QFileDialog.getSaveFileName)를 띄워 PNG 이미지 저장 경로를 받아줘.
2. Matplotlib Figure 객체의 savefig 함수를 사용해 현재 그려진 대시보드 차트 그림 자체를 고해상도(300 DPI) 이미지로 파일 출력 완료하는 최종 완성본 코드를 작성해줘."

[프로젝트 4] CRUD 및 AI 수질 예측 ,관제 대시보드

상황 부여: 로컬 SQLite3 데이터베이스에 연동하여 데이터 수동 저장(Insert) 및 행 삭제>Delete) 작업을 수행하고, DB 변경 시 실시간 지점별 평균 탁도 막대를 갱신(0.5 초과 시 빨간색 경고 바 노출)합니다. 나아가, DB 데이터를 판다스로 읽어 scikit-learn 머신러닝 객체에 동적으로 실시간 주입 학습(Fit)시킨 뒤, 예상 강수량 입력 시 미래 탁도 예측치(NTU)를 수학적으로 추산하여 알려주는 최고 사양의 스마트 대시보드를 구축합니다.

1단계: SQLite 연동 DB 테이블 생성 및 화면 추가/삭제 연계 (DB CRUD)

프롬프트 입력해보기:

"PyQt6, sqlite3를 사용하여 종합 모니터링 시스템을 개발하자.

1. DB 파일명은 kwater_monitoring.db로 하고 id(기본키), location(위치), rainfall(강수량), turbidity(탁도)를 가지는 TB_MONITORING 테이블이 없으면 자동 생성 후 초기 샘플 행들을 삽입해줘.
2. 좌측 패널에는 '위치, 강수량, 탁도' 입력창과 '데이터베이스에 저장' 버튼을 뒤서 DB에 INSERT하게 해줘.
3. 우측 상단에는 QTableWidgetItem와 '선택 행 데이터 삭제' 버튼을 두고, 버튼 클릭 시 해당 행의 고유 ID를 조회해 DB에서 DELETE 쿼리를 수행하게 해줘.
4. 삽입과 삭제가 일어나면 표 목록이 자동으로 리플래시 갱신되어 새로 보이는 코드를 조립해줘."

2단계: 실시간 차트 동기화 캔버스 탑재

프롬프트 입력해보기:

"이전 코드의 우측 패널 하단 구역에 Matplotlib 캔버스(FigureCanvasQTAgg)를 추가 임베딩해줘.

1. DB 입력이나 삭제가 일어날 때마다 refresh_dashboard 함수 내에서 자동으로 DB 데이터를 판다스 데이터프레임으로 다시 읽어와서, '지점별 평균 탁도'를 groupby로 연산하게 해줘.
2. 연산된 평균 탁도 값을 막대 그래프로 새로 그리도록 ax.clear()와 canvas.draw()를 연동해줘.
3. 평균 탁도 값이 0.5 NTU 기준치를 초과하는 지점은 빨간색, 0.5 NTU 이하인 안전 지점은 파란색 막대로 색칠이 다르게 표시되고, 0.5 NTU에 점선 경고 라인(axhline)까지 이식되도록 보강된 코드를 작성해줘."

3단계 (최종): Scikit-Learn 머신러닝 연동 및 실시간 AI 수질 예측기 결합

프롬프트 입력해보기:

"이전 코드의 좌측 패널 입력창 밑에 머신러닝 예측 구역을 추가해줘.

1. '예상 강수량 (mm) 입력' QLineEdit 입력창과 'AI 탁도 예측 실행' QPushButton 버튼, 결과를 표시할 QLabel 라벨을 세로 정렬 추가해줘.
2. 예측 버튼을 클릭하면, 현재 DB의 모든 'rainfall(강수량)'과 'turbidity(탁도)' 데이터 컬럼을 판다스로 추출해줘.
3. 추출한 데이터를 Scikit-Learn의 LinearRegression 선형 회귀 객체에 입력 변수(X)와 결과 변(y)으로 주입해 실시간으로 지도 학습(fit)시키고, 입력한 예상 강수량에 대해 예측한 탁도 수치(predict) 및 안전/주의 한글 상태 판정 결과를 라벨에 출력하는 최종 프로젝트 4 완성 코드를 작성해줘."